CS4246 Project Report

Chong Yew Mun Luke	e0905349	A0245763X
Panav Dua	e0912433	A0246189R
Wang Mingchuan	e0774727	A0240129N
Li Chengyue	e0694448	A0230326U

Abstract

This project explores the development of an AI agent capable of playing the classic Snake game using both reinforcement learning (RL) and classical heuristic search techniques. We focused on designing and evaluating a custom Snake environment to support the training and benchmarking of these agents. For the RL approach, we implemented and trained agents using Proximal Policy Optimisation (PPO), with an emphasis on optimising the reward structure using traditional methods as well as Large Language Models (LLMs). In parallel, we designed heuristic strategies to provide a baseline for comparison. The project highlights the trade-offs between RL and traditional methods in terms of adaptability, efficiency, and complexity.

Acknowledgements

Our main Snake game engine was built upon an open-sourced project by Safi available on Github. We used this as a foundation to create our own gym environment and testing pipeline.

1 Introduction

The game of Snake is a classic single-player game played on a 2D grid, where a snake navigates the environment in search of fruits. Consuming fruits increases the snake's length and score, while the game ends upon collision with walls or the snake's own body. Despite its simplicity, Snake provides a useful testbed for evaluating AI agent performance due to its clear objectives and gradually increasing difficulty.

In this project, we approach the problem of solving Snake using both classical heuristic search methods and reinforcement learning. While a relatively simple game like Snake can often be solved effectively with traditional methods, applying RL allows us to explore the broader challenges involved in training these RL agents. Specifically, we investigate how factors such as reward structure, model architecture, and hyperparameter choices impact training stability and performance. This also leads us to explore techniques for accelerating the iterative development and tuning of RL agents such as using LLMs.

1.1 Game Environment and Parameters

The Snake game engine was adapted from an open source snake implementation (Safi (2021)), which handles game logic involving fruit spawning and snake collisions. We chose to solve the 10×10 game. Each coloured frame is rendered as a $10\times10\times3$ numpy array. Fruits and the snake's initial location are randomly chosen. The snake has 4 actions to choose from at every frame. The expected rules regarding snake apply for eating and colliding with walls or itself. However we have an additional parameter which is health points (HP). The snake will lose 1 HP after every move, but will gain some back upon eating fruits. The snake will die at 0 HP and will win once the snake grows enough to fill the screen. We also allow ourselves to modify the snake initial HP and initial tail size to experiment with during RL training. The goal is to maximise the score (number of fruits eaten) without dying.

2 Classical Planning

In this section, we will use some classical heuristic algorithm to play the snake game. The game will be coded on a separate python file, before integrated into the main environment.

2.1 Agent, States, Actions and Goal

States: Location of snake head, Location of snake tail, Location of fruit
Actions: Up, Down, Left, Right
Goal: Snake move its head into the fruit (and eat it), whilst avoiding the walls and its own tail.

The states can be read from the main game environment. The heuristic agent will then build its own local environment and output the best action, which can then be fed into the main environment. To simplify the local environment, We will assume that only 1 fruit exists on the board each time. The goal of the agent is for the snake to grow and gain points, so no HP reduction will be implemented.

2.2 Heuristic Agent 1: Greedy Best-First Search

The first heuristic we will be using is the basic Greedy Best-First Search (GBFS). The snake will check its immediate next action, then choose the action that does not die and moves closest to the fruit. However, the agent will soon find itself getting trapped by its own tail as the fruit might be blocked by its own tail. Recording of the run can be viewed here. The agent dies rather fast. In 1000 runs, it survives an average score of about 20, or 20 fruits eaten.

2.3 Heuristic Agent 2: Greedy Best-First Search with Split Check

Improving upon our GBFS algorithm, we decided to implement an area check function so that the head of the snake does not enter an area that is too small and get itself trapped. As such, a BFS algorithm is added to make sure that the every move the snake takes, it does not split the map into two disjoint areas. The BFS search must be able to cover all the remaining blank tiles. In Figure 1, the snake head will turn left at the previous step following the GBFS algorithm. However, it will split the empty tiles into two. Thus, it went a step further down. However, after running the game several times, we realise that the snake will eventually have to split the map into two. Given this scenario, we made the snake enter the area with a larger number of empty tiles to increase the chance of surviving.
Despite this, the agent will still die. As the tail length increases, it will eventually run out of space as the tail end is blocked by its own body. As seen in Figure 2, the snake will eventually die since the tail is on top. Recording of the run can be viewed here. The agent makes great improvements from the Heuristic Agent 1. In 1000 runs, it survives an average score of about 47, or 47 fruits eaten.

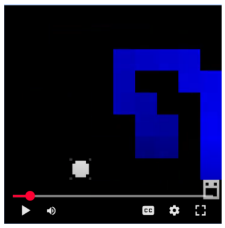


Figure 1: Agent avoids splitting the empty tiles

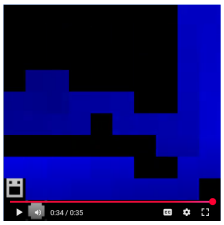


Figure 2: Agent dies as space runs out

2.4 Heuristic Agent 3: Greedy Best-First Search with Hamiltonian Cycle

Improving upon Heuristic Agent 2, we added another condition that the snake's tail end must be reachable from its head. This makes ensures that there exists a "Hamiltonian Cycle", if we trace the snake from its head to tail end, then to its head again over the empty tiles. However, it is not a full Hamiltonain Cycle since it might not cover every empty tile.
Towards the late game, the agent will start chasing its tail, whilst going after the fruit whenever the condition allows. This ensures that the snake never dies from getting trapped. However, it is still not perfect as the greedy instinct will result in some of the empty space to be pushed towards the edge and there will be no space to eat the fruit. Recording of the run can be viewed here.
This heuristic greatly improves the score. In 1000 runs, it survives an average score of about 84, completing the game 24 times.

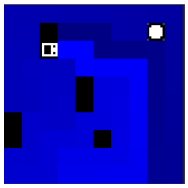


Figure 3: Agent continuously chase its tail, whilst going for fruit

3 Reinforcement Learning

3.1 Algorithm and Training Pipeline

For RL training, we used the Stable-Baselines3 (SB3) (Raffin et al.) library for their RL algorithms which simplified the training process. The algorithm we chose was PPO for its out-of-the-box performance, and preliminary testing showed poorer performance for algorithms such as Advantage Actor Critic (A2C).

We adapted our game environment to suit the Gymnasium (Towers et al.) interface, which is a maintained fork of OpenAI's Gym library. The observation or state space which serves as the input for the agent is the 10×10×3 grid array, which contained information about the snake's location and fruit location. The agent is not given any other information. The action space includes the 4 movements (Up, Down, Left, Right). An episode terminates whenever the snake head transitions into a cell containing a tail, or goes out of bounds. The episode also terminates when the snake fills the screen. To speed up trajectory collection, we also ran the training in a vectorised environment of 32 parallel environments.

Due to the number of game parameters and hyperparameters, these settings were saved in configuration files to be edited quickly and be saved for future testing. The settings will be loaded from these files during the training session.

3.2 Optimising Network Architecture and Size

Our preliminary testing showed that model size heavily affects not just training performance but also training time. Too large a model results in slower training as there will be a greater amount of parameters to change, but too few parameters and the model will fail to capture sufficient information to perform well. By default, the model includes a feature extractor network (FEN) and an actor-critic network (ACN). The FEN takes in an image as input and consists of 3 convolutional layers, with the first layer having 32 filters and the rest, 64 filters. This is followed by a linear layer which returns a flattened feature vector. This is then passed into the ACN for policy and value prediction. The ACN includes the policy and value function network, which consists of a single linear layer with the appropriate output size.

In preliminary testing, we increased the model size by artificially scaling up the image by 64 times. This changed the input size to 640×640×3. The resultant model was too large, which resulted in a prolonged training time which was unsuitable for quick iteration. Furthermore, the resulting performance was unsatisfactory. However, scaling up was still necessary for adding model complexity, so a scale of 4 was chosen.

Further testing revealed that the default architecture had limitations, even with a close to optimal reward structure. The feature extractor was changed to having only 2 convolutional layers instead of 3, with each layer having 32 and 64 filters respectively. Furthermore, the extracted features were passed through an additional 2-layer feed forward network before being passed into the actor-critic network. This is an overall increase in model size, which significantly improved training performance. This improvement was captured when training on a 6 × 6 grid. Where the old architecture could only achieve an average score of 15.7/35 over 10 episodes (sample run), the agent trained with the new architecture obtains an average score of 34/35 (sample run), which shows that the new architecture is an improvement.

Our preliminary tests show that modifying the actor-critic network did not affect the training in a positive way. We attempted to increase the number of layers and detach the policy and value function network. We posit that there will be some optimal architecture, but further testing is required to find it.

Figures comparing the default and modified model architecture can be found in the appendix.

3.3 Optimising Reward Structure

The bulk of the RL portion of this project was dedicated to finding the optimal reward structure. We tested a few methods which in principle allows for quick iteration. Since these methods are not environment

specific, they can be extended to RL problems in general.

3.3.1 Manual Testing

Due to time constraints, most of the manual testing was done for the game parameters rather than algorithm hyperparameters such as learning rate or clip range. For most of our tests, unless otherwise stated, the default choices were used for of these hyperparameters as they worked well during preliminary testing. The exact values are shown in Table 1. The game parameters and rewards that were extensively tested are in Table 2.

Table 1: Hyperparameters

Hyperparameter	**Value**
n_steps	128
timesteps	4,000,000
batch size	2,048
learning rate	0.0003
clip range	0.2
entropy coefficient	0

Table 2: Game Parameters and Reward Events

Game Parameters	**Reward Events**
Initial HP	Fruits Eaten
Initial tail length	Death
Num. fruits	Idle Penalty
Fruit heal	Win Game
Episode length	–

We hypothesized that setting a low initial HP and a high fruit heal value would encourage the agent to seek out fruits more aggressively, as survival would depend on frequent healing. This setup was expected to promote learning that fruits provides high reward. However, after testing various configurations of initial HP values [10,40,100,1000], we observed minimal impact on the agent's performance. We posit that this is because there is only a weak link between the snake HP and the rewards gained. The agent receives negative rewards for deaths, including due to losing all HP. However, the fact that eating fruits could prolong the lifespan of the snake is hidden behind the game mechanics and not explicitly shown to the agent. Therefore, it could be more difficult for the agent to learn this connection, beyond the direct rewards given to the snake for eating fruits. From this, we conclude that it is difficult to do reward shaping by simply changing the game mechanics. It is better to explicitly alter the rewards received.

A longer initial tail length would lead to a higher chance of the snake colliding with itself, so we hypothesized that by setting a longer tail length, the agent would more quickly learn to avoid collisions with its body. However, once again, while the agent did learn to avoid its body, an agent that starts with low initial tail length also does the same with around the same training time. Unless otherwise stated, we set the initial tail length to 4.

The episode length/horizon parameter controls the maximum episode length before artificial termination during RL training even if a terminal state was not reached. While in principle setting the horizon to be short should encourage the agent to seek rewards quicker, we found that an increase in this parameter leads to better performance. The agent tended to learn long looping strategies when the horizon is 1000 steps but fails to learn significantly when the horizon is set to 50. This could be due to inefficiencies in our rewards where the agent is not able to learn from the reward set given. However, due to time constraints we were not able to test this extensively. For our final models, we trained them using a horizon of 1000. While they are able to achieve higher rewards, they do not seek out fruits in the most efficient way possible.

The number of fruits did have an effect the training performance and was the primary target for the curriculum learning section. We defer our analysis to that section.

For the reward set, we initially wanted to have a small negative reward for every idle step taken without getting fruits, as this may encourage fruit seeking without wasting steps. However, if the value is too negative, then the agent will simply learn that terminating the episode early is always better, so a good balance must be struck. We also manually looked for a good ratio of rewards for getting fruits and dying to optimize for fast learning. The final ratio after manual testing is given in Table 3.

Table 3: Manual reward assignments

Event	**Value**
Fruits Eaten	+20
Death	-10
Idle Penalty	-0.4
Win Game	+1

3.3.2 Genetic Algorithm Approach

While we were able to obtain passable results with manual testing, we wanted to experiment with methods to speed up the search for the optimal or close to optimal reward set. One clear option would have been to do a full grid search over chosen values, but since we used real-valued rewards, this could be more time consuming and less precise. We decided to use a genetic algorithm, which could more easily narrow down good features and spend more time testing the best combinations. We used PyGAD (Gad (2023)) for their algorithm and searched optimal reward values for Fruit Eaten, Death and Idle Penalty.

For each reward set which the genetic algorithm generates, we run PPO training for 400,000 timesteps before evaluating its performance. This at least allows the agent to sufficiently learn without spending too much time per iteration. We set the population size and number of generations to be 10 and 20 respectively, which results in about 200 combinations tested, which was a sufficient amount for our purposes. The rewards are summarized in Table 4.

Table 4: Reward optimization via genetic algorithm

Event	**Search Range (low, high)**	**Best Value**
Fruit Eaten	$(10.0,\ 100.0)$	+37.4
Death	$(-100.0,\ -10.0)$	-68.2
Idle Penalty	$(-3.0,\ 0.5)$	-1.43

The resulting reward set performed decently during further testing and was used for subsequent tests. An interesting observation was that the magnitude of reward for dying is twice the amount of getting fruits, which could be a good guideline to follow for this particular environment. Even if the genetic algorithm did not return the optimal reward set, we find it valuable for understanding the general range of values which the rewards should take. When training in a new environment with inadequate domain knowledge, a genetic algorithm would be a good starting point for reward shaping.

3.3.3 Curriculum Learning

The reward sets chosen previously seemed to work well. However, the agent is still unable to complete the board and typically dies with a score of about 10. We explored the idea of varying the reward set during the training process to allow it to focus on sub-tasks like getting fruits or not colliding with itself/walls. For this, we incorporated the idea of curriculum learning by tweaking the rewards set and the game parameters.

Initially, we ran tests while varying the initial HP and initial tail length. However, these did not yield better results. Furthermore, choosing the right set of rewards for eating fruits or dying was difficult as there were too many moving parts. Tests which change these variables did not show much difference than without using curriculum learning at all.

Ultimately, the change that did show results was the change to the number of fruits shown on the board at any given frame. By starting off with a larger number of fruits and slowly decreasing, the agent seems to learn better than without. This is likely due to sparse rewards being an issue during the training. So this made rewards denser, which incentivised the agent to go for the fruits. Figure 4 shows the learning curves for training with and without curriculum learning for the number of fruits. The blue line represents the training without curriculum learning while the rest use curriculum learning. We show multiple runs to highlight that despite working better, the average training performance with the same conditions (demo) is not close to our best performing model (demo). The latter averages a score of **76.4/95**, which is close to a perfect score. It was difficult to reproduce the performance of this model even with same conditions, and its performance could not be improved with further training, as it only resulted in catastrophic forgetting.

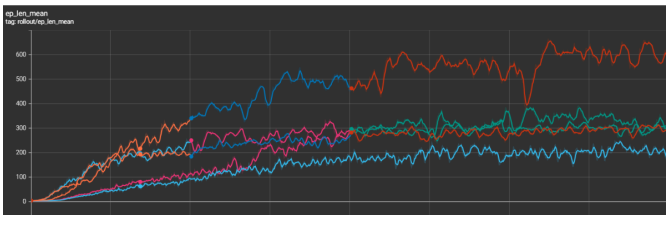


Figure 4: Curriculum Learning Comparison: Without (Blue) | Best (Top) | Average (Middle)

3.4 LLM-Guided Reward Shaping

To address the issue of training plateaus and suboptimal learning trajectories in reinforcement learning for Snake, we implemented a reward tuning pipeline guided by Large Language Models (LLMs), inspired by the methodology of Kwon et al. (2023). In the original work, an LLM is used to iteratively revise a robot manipulation reward function based on human-provided behavioral summaries, showing that LLM-derived reward updates can match or outperform hand-tuned baselines while improving sample efficiency.

Our adaptation follows a similar cycle tailored to the Snake environment. The overall loop comprises four stages: RL training under a fixed reward configuration, collection of performance metrics, reward update decision by the LLM, and configuration update. This process repeats to progressively optimize performance.

3.4.1 Reward Space Augmentation

We expanded the reward space to enable the LLM to target specific behavioral patterns causing local optima. Plateaus in Snake often arise from different failure modes (e.g., circular movement, wall avoidance loops). Our extended reward set allows the LLM to control more levers, providing fine-grained influence. Table 5 outlines each component.

Table 5: Extended Reward Components for Snake Training

Reward Name	**Purpose**
`fruit_reward`	Incentivizes eating fruits
`death_penalty`	Penalizes dying
`step_penalty`	Discourages passive behavior
`won_reward`	Bonus for winning the game
`center_bonus`	Encourages movement to center
`loop_penalty`	Penalizes repetitive trajectories
`wall_follow_penalty`	Penalizes hugging walls
`exploration_bonus`	Rewards novel area visitation
`consecutive_fruit_bonus`	Rewards fruit collection streaks
`distance_reduction_reward`	Encourages greedy movement to fruit
`wall_avoidance_bonus`	Rewards avoiding walls

3.4.2 Exposed Training Configurations

In addition to rewards, the LLM is given access to training and environment parameters for broader influence. These parameters impact learning stability and difficulty (Table 6).

Table 6: LLM-Configurable Training Parameters

Parameter	**Effect**
`entropy_coef`	Controls exploration level
`clip_range`	Affects PPO update stability
`num_fruits`	Changes fruit density
`walls_config`	Modifies map complexity
`max_timesteps`	Alters episode length (influences horizon)

3.4.3 Performance Metrics Collection

After each training cycle, we compute a summary of agent behavior to be passed as LLM context. These metrics reflect key indicators for diagnosing learning quality or stagnation:

Table 7: Snake Agent Performance Metrics

Metric	Description
`avg_episode_length`	Steps per episode
`avg_fruit_per_episode`	Mean fruits consumed
`avg_max_snake_length`	Maximum body length
`avg_map_coverage_pct`	Proportion of explored map
`looping_rate_pct`	Frequency of cyclic paths
`avg_action_entropy`	Policy entropy (diversity of decisions)

Initially, these metrics were in raw text form. However, this failed to capture trends or temporal dynamics essential for diagnosing training progress. As such, we incorporated time-series visualizations (e.g., plots over reward iterations), which are submitted to the LLM alongside the textual context.

3.4.4 LLM Prompt Design

To guide the reward tuning process, we designed a structured prompt for GPT-4o, the LLM chosen. The prompt includes four key components:

- **Current reward configuration:** Provided in JSON format.
- **Reward explanation:** A brief description of how each reward signal contributes to the agent's behavior.
- **Performance metrics:** Numerical indicators such as average fruits collected, episode length, and action entropy.
- **Visual trend plots:** Graphical summaries of the metrics across previous iterations to highlight stagnation or improvement.

The LLM is tasked with analyzing this context and deciding whether a reward update is necessary. If training appears to be plateauing or regressing, the LLM proposes incremental adjustments to specific reward components (within $\sim$10–50% of current values). If learning is still ongoing or highly unstable, the LLM may recommend no changes.

Example Prompt Template:

```
You are an expert in reinforcement learning reward shaping. Your task is to optimize
the reward function for a Snake game agent based on recent performance data and
trends.

Metric Trends: Refer to the attached line plots showing changes in key metrics
over time.
Current Metrics: {metrics_str}
Reward Definitions: {rewards_explanation}
Current Reward Function:

{ ... JSON-formatted reward configuration ... }

Task: Determine if reward adjustments are needed. If yes, return an updated
reward configuration with proportional changes (±10-50%). If no changes are needed,
return the current configuration unchanged. Only output the JSON.
```

3.4.5 Iterative Optimization Loop

After the LLM proposes a new reward configuration, we apply it and keep training. This closes the loop:

`[Train RL Agent` $\rightarrow$ `Collect Metrics` $\rightarrow$ `LLM Analysis` $\rightarrow$ `Update Rewards]`

This cycle repeats until the agent either converges or reaches a stable, high-performing plateau. We observed that this method was particularly effective at breaking training stagnation and helped adaptively steer the learning process.

3.5 Results and Limitations of LLM-Guided Tuning

Despite its theoretical appeal, our implementation of LLM-guided reward tuning yielded largely negative results in practice. Instead of accelerating training, the intervention frequently destabilized the learning process and introduced erratic behavior.

3.5.1 Training Destabilization

Each time the LLM swapped out the reward function, the RL agent exhibited a sharp deviation from its prior learning trajectory. Rather than incrementally improving, the agent often regressed in performance. This disruption suggests that the LLM lacks sufficient priors or reasoning capacity for nuanced reward shaping in this context, or alternatively, that it requires fine-tuning for this specific task.

Observed Effect: Destabilization After Reward Updates



Figure 5: Training curves after LLM-updated reward function. Performance drops significantly.

3.5.2 Catastrophic Forgetting

A particularly damaging phenomenon we observed is *catastrophic forgetting*—after the LLM proposes a new reward configuration, the agent often forgets previously learned useful behaviors. This leads to drastic and sudden drops in performance, even in cases where the new rewards are semantically reasonable.

Observed Effect: Agent Forgets Previous Strategies



Figure 6: Agent performance after reward change. Sudden loss of prior good behaviors.

3.5.3 Rare Positive Outcomes

There were isolated cases where the LLM correctly adjusted training parameters, resulting in substantial performance gains. For instance, in one run, the LLM modified the PPO entropy coefficient to 0.05. This led to a significant improvement, successfully breaking a training plateau that all other runs failed to surpass. However, such successes were extremely rare and appear stochastic rather than systematic.

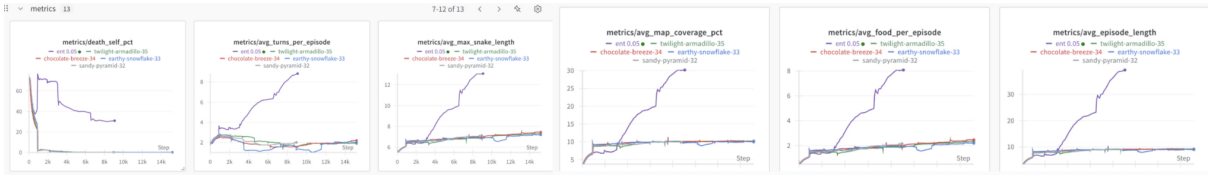


Figure 7: A rare successful run where LLM-set entropy coefficient (0.05) led to performance breakthrough.

References

Gad, A. F. (2023). Pygad: An intuitive genetic algorithm python library. *Multimedia Tools and Applications*, pages 1–14.

Kwon, M., Xie, S. M., Bullard, K., and Sadigh, D. (2023). Reward design with language models.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., and Dormann, N. (2021). Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8.

Safi, J. (2021). snake.py — snake game environment. https://github.com/safijari/jack-of-some-rl-journey/blob/master/snake.py. Accessed: 2025-03-19.

Towers, M., Kwiatkowski, A., Terry, J. K., Balis, J. U., de Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., Perez-Vicente, R., Pierré, A., Schulhoff, S., Tai, J. J., Tan, H. J. S., and Younis, O. G. (2024). Gymnasium: A standard interface for reinforcement learning environments. https://arxiv.org/abs/2407.17032. Version from the Farama Foundation GitHub: https://github.com/Farama-Foundation/Gymnasium.

4 Appendix

Link to Code

- Heuristic agent tests
- RL tests
  - Contains RL training tensorboard logs
  - Run eval.py file for the final model evaluation
- LLM reward shaping tests

RL Training Parameters and Results

MODEL	2	3	3.1	3.11	3.111
Reward for doing ntg	-2	0	0	0	0
Reward for dying	-10	-10	-10	-10	-10
Reward for getting food	5	5	5	5	5
Reward for winning	1	1	1	1	1
Additional reward (+)	0	0	0	max(hp/length, 10)	min(hp/length, 10)
Reward scaling	0.01	0.01	0.01	0.01	0.01
fruits	10	10	10	10	10
initial hp	10	10	10	10	10
hp decay	1	1	1	1	1
fruit heal	10	10	10	10	10
initial tail size	2	2	2	2	2
perspective					
HYPERPARAMS					
n_steps	128	128	1024	1024	1024
timesteps	100000	100000	100000/1mil	100000	100000
truncation steps	50	50	50	50	50
batch size					
learning rate	0.0003	0.0003	0.0003	0.0003	0.0003
clip range					
policy					
Notes	kills itself asap, avoids food	higher variance for rew/len	same general shape, slightly better given same timesteps	mean rewards not increasing, staying the same, low average length	similar means as before, but slightly higher
		speed of convergence lower, more steps could be used, but upward trend	less variance than when fewer n_steps	prefers to go in circles	doesnt seem to be performing better, still not valuing food that much
		learned to not go out of bounds, but not actively seeking out food	when increased to 1mil timesteps, cant seem to break through the 0 reward area	seems like providing positive reward for doing nothing is not great	
				max is wrong, rip, should be min	

RL Training Parameters and Results

MODEL	3.112	3.12	3.121	3.13	3.131
Reward for doing ntg	0	0	0	0	-1
Reward for dying	-10	-10	-10	-10	-10
Reward for getting food	5	5	5	5	5
Reward for winning	1	1	1	1	1
Additional reward (+)	min(hp-init_hp/length, 10)	0	0	0	0
Reward scaling	0.01	0.1	10	0.01	0.01
fruits	10	10	10	10	10
initial hp	10	10	10	10	10
hp decay	1	1	1	1	1
fruit heal	10	10	10	10	10
initial tail size	2	2	2	10	10
perspective					
HYPERPARAMS					
n_steps	1024	1024	1024	1024	1024
timesteps	100000	100000	100000	100000/1mil	200k
truncation steps	50	50	50	50	50
batch size					
learning rate	0.0003	0.0003	0.0003	0.0003	0.0003
clip range					
policy					
Notes	much lower averages and slower improvement	perhaps the rewards are too small around 0 which caused vanishsing gradients, try scaling to larger value	not great	should train more on the same model	kills itself
		doesnt seem to help, 1/100 seems to be more optimal given the learning rate		look at policy gradient loss	likely food is not attractive enough or issue with the observation (cant see food)
				appears to be slower learning, but could increase past 1 mil steps	
				very slow improvement after 500k, maybe could benfit from tuning learning rate	

RL Training Parameters and Results

MODEL	3.132	3.1321	3.1322	3.1323	3.1324
Reward for doing ntg	0	0	0	-0.1	-0.3
Reward for dying	-10	-10	-10	-10	-10
Reward for getting food	10	20	20	20	20
Reward for winning	1	1	1	1	1
Additional reward (+)	0	0	0	0	0
Reward scaling	0.01	0.01	0.01	0.01	0.01
fruits	10	10	10	10	10
initial hp	10	10	10	10	10
hp decay	1	1	1	1	1
fruit heal	10	10	10	10	10
initial tail size	10	10	10	10	10
perspective					
HYPERPARAMS					
n_steps	1024	1024	1024	1024	1024
timesteps		500k	100000/1mil	100k/1mil	100k/1mil
truncation steps	50	50	50	50	50
batch size					
learning rate	0.0003	0.0003	0.00025	0.0003	0.0003
clip range					
policy					
Notes	snake seems to always have low probability of choosing right, highest probability of choosing stay, no matter the state	very similar length trajectory as 3.13, higher rewards simply due to reward definition	lower variance, not enough change after 300k	training slows down but not as much as before	seems like there is a threshold where the average rewards increase faster
	no learning or local optimum reached after 300k steps. not much change to the action distribution		didnt change clip fraction anyway, probably not worth changing this much, at least for the first 300k steps	good foundation for continuing training	
	policy loss at 0				
	maybe change clip range for this				

RL Training Parameters and Results

MODEL	3.1325	3.1326	3.2	3.3	3.31
Reward for doing ntg	-0.5	-0.4	-0.4	-0.4	-0.4
Reward for dying	-10	-10	-10	-10	-10
Reward for getting food	20	20	20	20	20
Reward for winning	1	1	1	1	1
Additional reward (+)	0	0	0	0	0
Reward scaling	0.01	0.01	0.01	0.01	0.01
fruits	10	10	10	10	1
initial hp	10	10	10	10	40
hp decay	1	1	1	1	1
fruit heal	10	10	10	10	20
initial tail size	10	10	10	10	2
perspective			first	first	first
HYPERPARAMS					
n_steps	1024	1024	128	128	128
timesteps	100k/1mil	100k/1mil	10*1mil	10*1mil	10*1mil
truncation steps	50	50	50	50	50
batch size			2048	2048	2048
learning rate	0.0003	0.0003			
clip range					
policy			multi	cnn	cnn
Notes	rewards 0, policy converges to [('stay', 0.838), ('left', 0.0), ('right', 0.162)] for all states	about the same performance as the -0.3 case	much higher rewards, learns to occupy one corner, but still dies by being trapped	Changed policy from multiinput to pure cnn policy, without the extra vector	doesnt know how to collect fruits closer to corners, goes in a loop (opposite direction). But it is able to generally collect fruits in other locations
				most action probabilities are 1, so the agent is quite certain with its moves	
				seems to mean that it converges or learns faster, likely due to reduced complexity	

RL Training Parameters and Results

MODEL	3.32	3.34	4.0a	4.0b	4.0c
Reward for doing ntg	-0.4	-0.4	-0.4	-0.4	-0.4
Reward for dying	-10	-10	-40	-10	-10
Reward for getting food	20	20	20	40	20
Reward for winning	1	1	1	1	1
Additional reward (+)	0	0	0	0	0
Reward scaling	0.01	0.01	0.01	0.01	0.01
fruits	1	1	1	5	1
initial hp	1000	40	100	40	40
hp decay	1	1	1	1	1
fruit heal	20	20	20	20	20
initial tail size	2	2	20	4	4
perspective	first	third	third	third	third
HYPERPARAMS					
n_steps	128	128	128	128	128
timesteps	10*1mil	10*1mil	500k	500k	1mil
truncation steps	50	50	50	50	50
batch size	2048	2048	2048	2048	2048
learning rate					
clip range					
policy		cnn	cnn	cnn	cnn
Notes	sparser rewards?	changed actions from 3 to 4	train the snake not to collide into the walls or itself, punish death greatly	reward the snake for getting food more, start with lower hp to encourage food seeking	back to default rewards for final training
	similar to 3.31, might as well use that	just reaches a different fixed point, but doesnt have the issue of not being able to see corners			train this one for longer
					this set doesnt feel good, doesnt go for food and still runs into walls

RL Training Parameters and Results

MODEL	4.1abc	4.2abc
Reward for doing ntg	-0.4	-0.4
Reward for dying	-10	-10
Reward for getting food	20	20
Reward for winning	1	1
Additional reward (+)	0	0
Reward scaling	0.01	0.01
fruits	10/5/1	10/5/1
initial hp	40	40
hp decay	1	1
fruit heal	20	20
initial tail size	4	4
perspective	third	third
HYPERPARAMS		
n_steps	128	128
timesteps	2mil	1mil
truncation steps	50	50
batch size	2048	2048
learning rate		
clip range		
policy	cnn	cnn
Notes	try reducing the number of fruits along the training	grid size 6
	hypothesis: trying to maximise reward and avoiding death are 2 separate goals, but achieving the first goal happens to also look like the second	does almost solve it, only 1 away
	sudden sharp increase in performance during the 1.8 mil mark.	used custom feature extractor
	Doesnt carry that performance into evaluation, still cannot perform well, but does with 10 fruits	

RL Training Parameters and Results

MODEL	4.3abc	4.4abc	4.4e	4.4f
Reward for doing ntg	-0.4			
Reward for dying	-10			
Reward for getting food	20			
Reward for winning	1			
Additional reward (+)	0			
Reward scaling	0.01			
fruits	5/3/1			
initial hp	40		100	100
hp decay	1			
fruit heal	20			
initial tail size	4			
perspective	third			
HYPERPARAMS				
n_steps	128			512
timesteps	4 mil total	8 mil total		
truncation steps	50			
batch size	2048			
learning rate				
clip range				
policy	cnn			
Notes	much better performance. best to date, training for 4 mil steps total allows it to survive for half the board		massive loss in performance halfway	